Request Dispatching and forward() vs include()

Request dispatching in servlets allows you to forward or include content from one servlet to another. This can be done using the RequestDispatcher interface. The two primary methods provided by RequestDispatcher are forward() and include().

1. RequestDispatcher Interface

- RequestDispatcher is used to dispatch a request from one servlet to another resource (another servlet, JSP, or HTML) on the server.
- **There are two main methods:**
  - **forward(request, response):** Forwards the request from one servlet to another resource on the server.
  - **include(request, response):** Includes the content of another resource (servlet, JSP, HTML) in the response.

2. Forwarding Requests

Important Points:

- **Control Transfer:** The control is completely transferred to another resource (like another servlet) and the client will not know about this change.
- **Single Response:** Only the forwarded servlet's response is sent back to the client. Any content generated before the forward() call is discarded.

forward() in your FirstServletApp:

```
@WebServlet("/firstservlet")
public class FirstServletApp extends HttpServlet {

        public FirstServletApp() {
        System.out.println("Internally constructor called");
        }

        public void doPost(HttpServletRequest request,
HttpServletResponse response) throws ServletException, IOException
{
        String name = request.getParameter("uname");
        String city = request.getParameter("ucity");
        PrintWriter pw = response.getWriter();

    //This code will not add a response to the output
        pw.println("Before dispatch");

        // Forwarding the request to SecondServletApp
        RequestDispatcher rdsp =
request.getRequestDispatcher("/SecondServletApp");
       rdsp.forward(request, response); // Control is transferred to
SecondServletApp

        // This code will not execute as the control is transferred to
SecondServletApp
        pw.println("Name from 1st : " + name);
        pw.println("City from 1st: " + city);
```

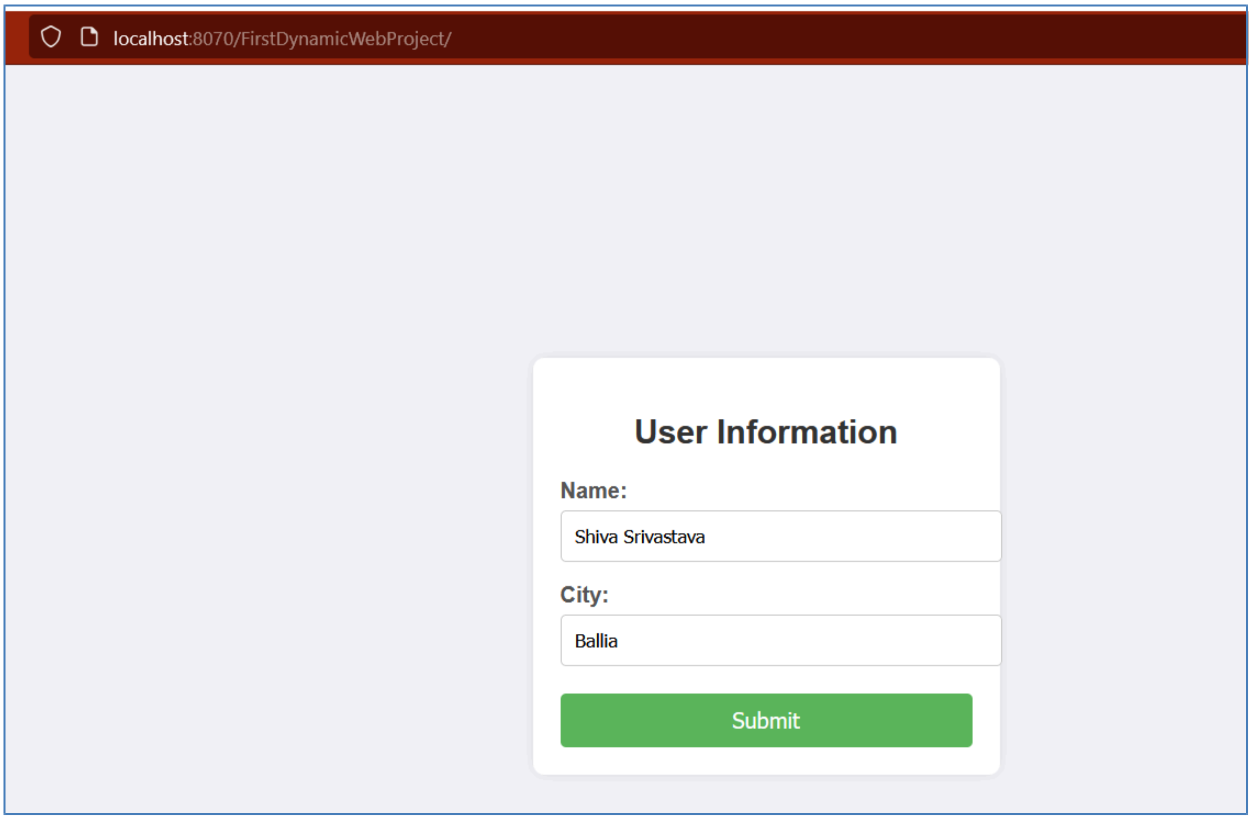
```
    System.out.println("execution come here");
        pw.close();
        }
}
```

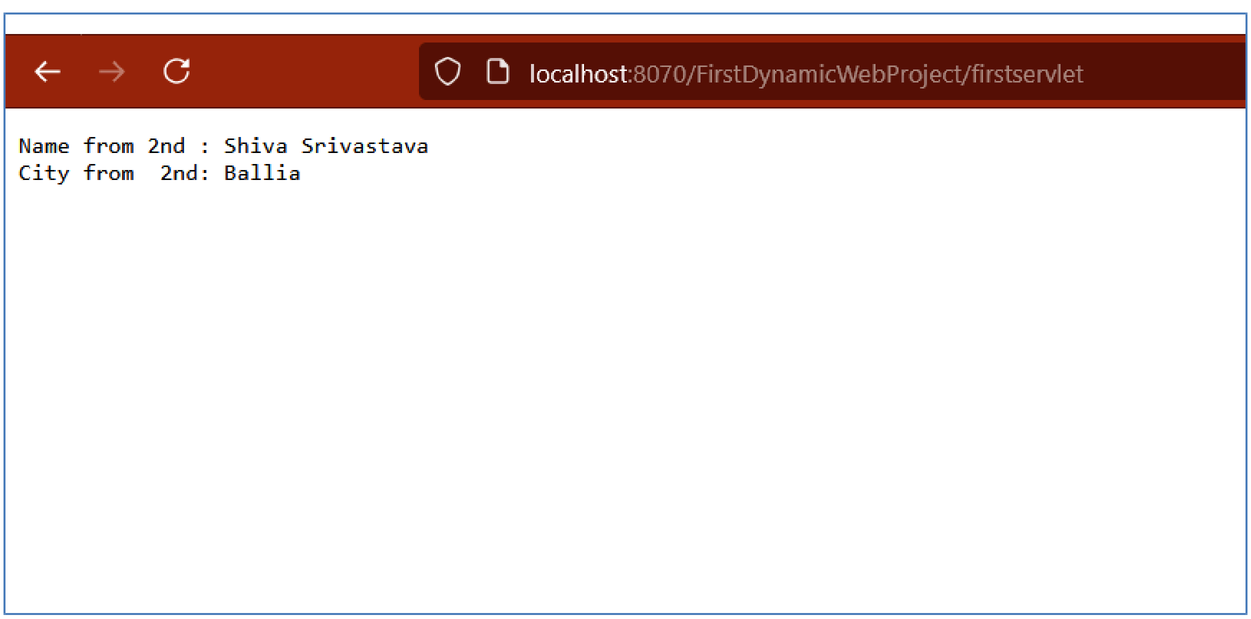
And the SecondServletApp:

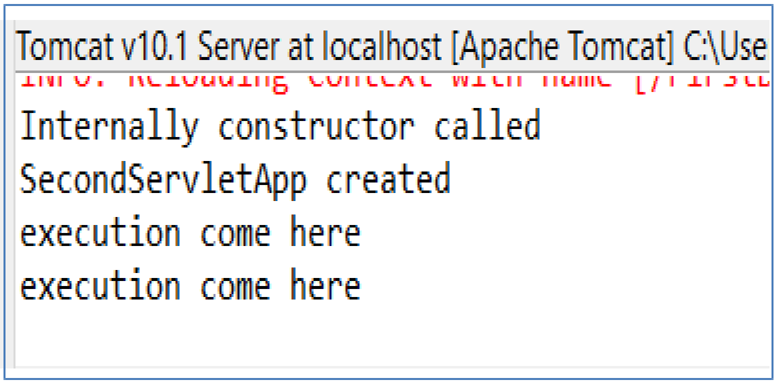
```
@WebServlet("/SecondServletApp")
public class SecondServletApp extends HttpServlet {
        private static final long serialVersionUID = 1L;

        public SecondServletApp() {
       System.out.println("SecondServletApp created");
        }

        public void service(HttpServletRequest request,
HttpServletResponse response) throws ServletException, IOException
{
        String name = request.getParameter("uname");
        String city = request.getParameter("ucity");

        PrintWriter pw = response.getWriter();
        pw.println("Name from 2nd : " + name);
        pw.println("City from 2nd: " + city);

        pw.close();
        }
```

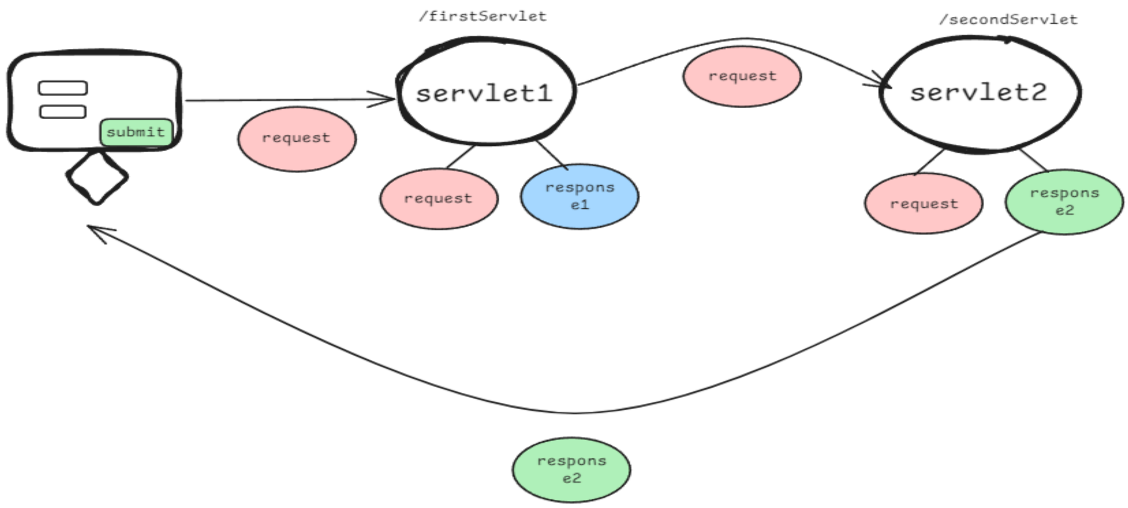
```
}
```

Output:


localhost:8070/FirstDynamicWebProject/
User Information
Name:
Shiva Srivastava
City:
Ballia
Submit


localhost:8070/FirstDynamicWebProject/firstservlet
Name from 2nd : Shiva Srivastava
City from 2nd: Ballia





Forward dispatching

- The response will only show content from SecondServletApp, as the control is completely transferred there.
- Any content written to the response (using PrintWriter) before the forward() call is discarded and does not appear in the final output. This is because forward() transfers control to another resource, and the response buffer is cleared before the forwarding occurs.

3. Including Requests

Important Points:

- **Control Retention:** The control returns back to the original servlet after including the content of another resource.
- **Combined Response:** The response from the included resource is combined with the response of the original servlet.

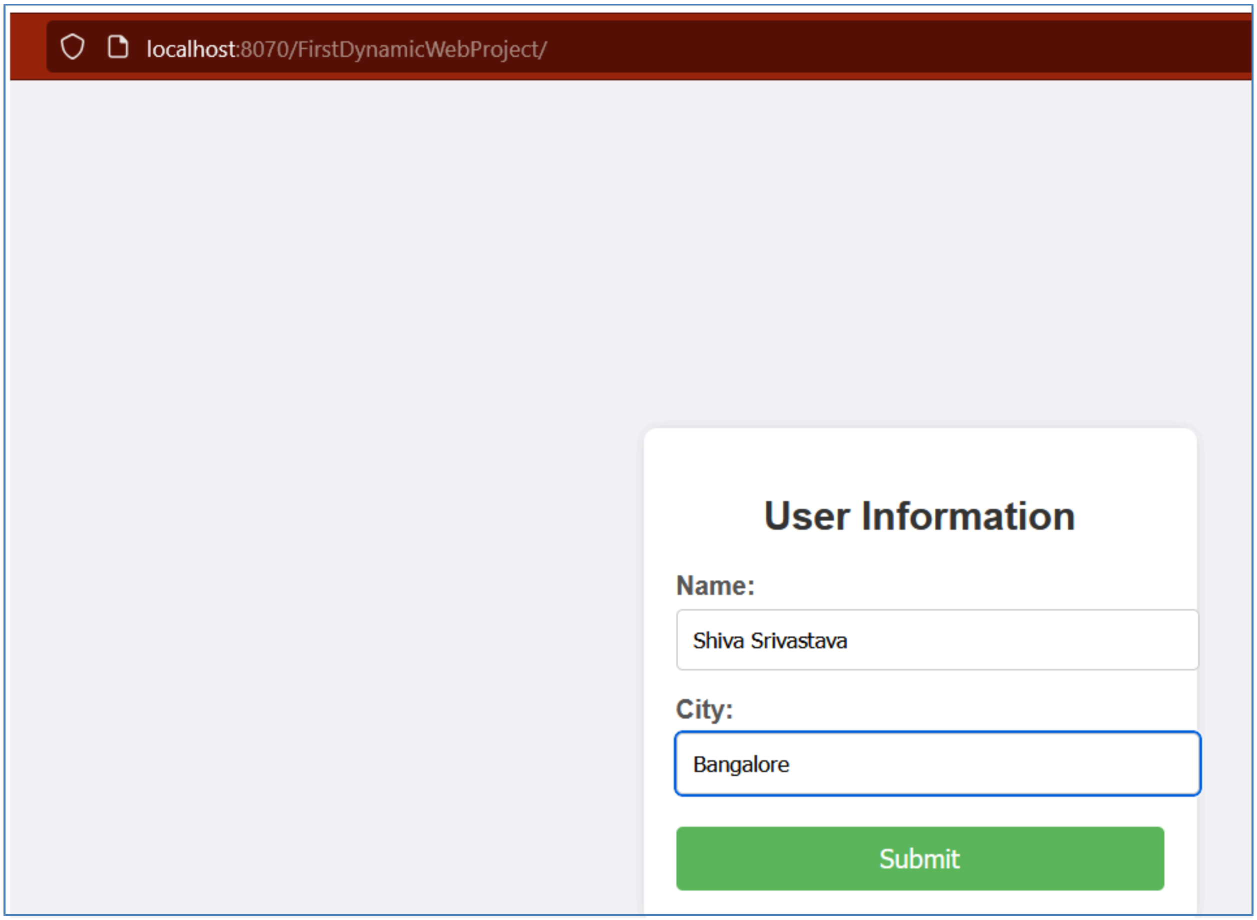
FirstServletApp to use include():

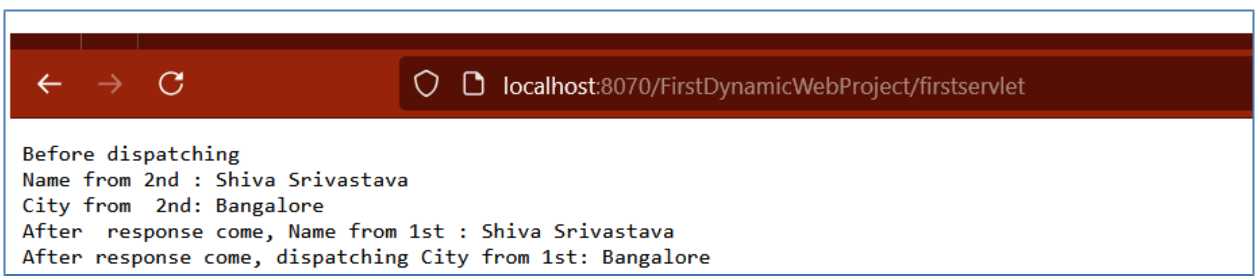
```
@WebServlet("/firstservlet")
public class FirstServletApp extends HttpServlet {

        public FirstServletApp() {
 System.out.println("Internally constructor called");
        }

        public void doPost(HttpServletRequest request,
HttpServletResponse response) throws ServletException, IOException
{
        String name = request.getParameter("uname");
        String city = request.getParameter("ucity");
        PrintWriter pw = response.getWriter();

        pw.println("Before dispatch");

        // Including the response from SecondServletApp
        RequestDispatcher rdsp =
request.getRequestDispatcher("/SecondServletApp");
       rdsp.include(request, response); // Control is temporarily
```

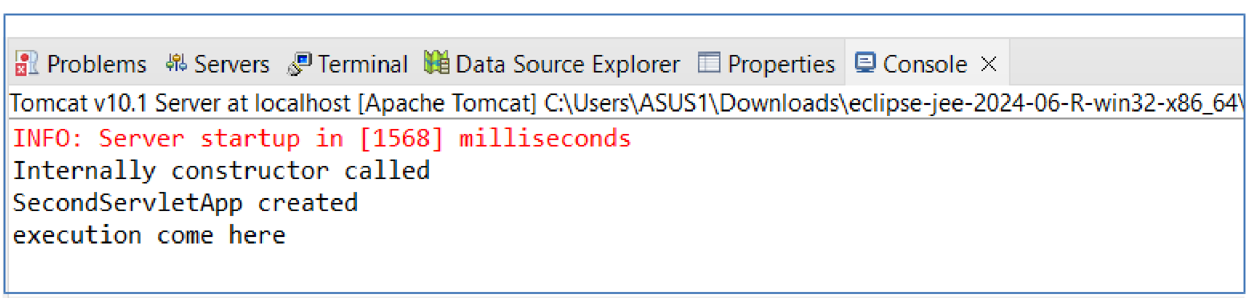
```
transferred to SecondServletApp and returns back

// This code will execute after including the SecondServletApp response
        pw.println("Name from 1st : " + name);
        pw.println("City from 1st: " + city);

  System.out.println("execution come here");
        pw.close();
        }
}
```

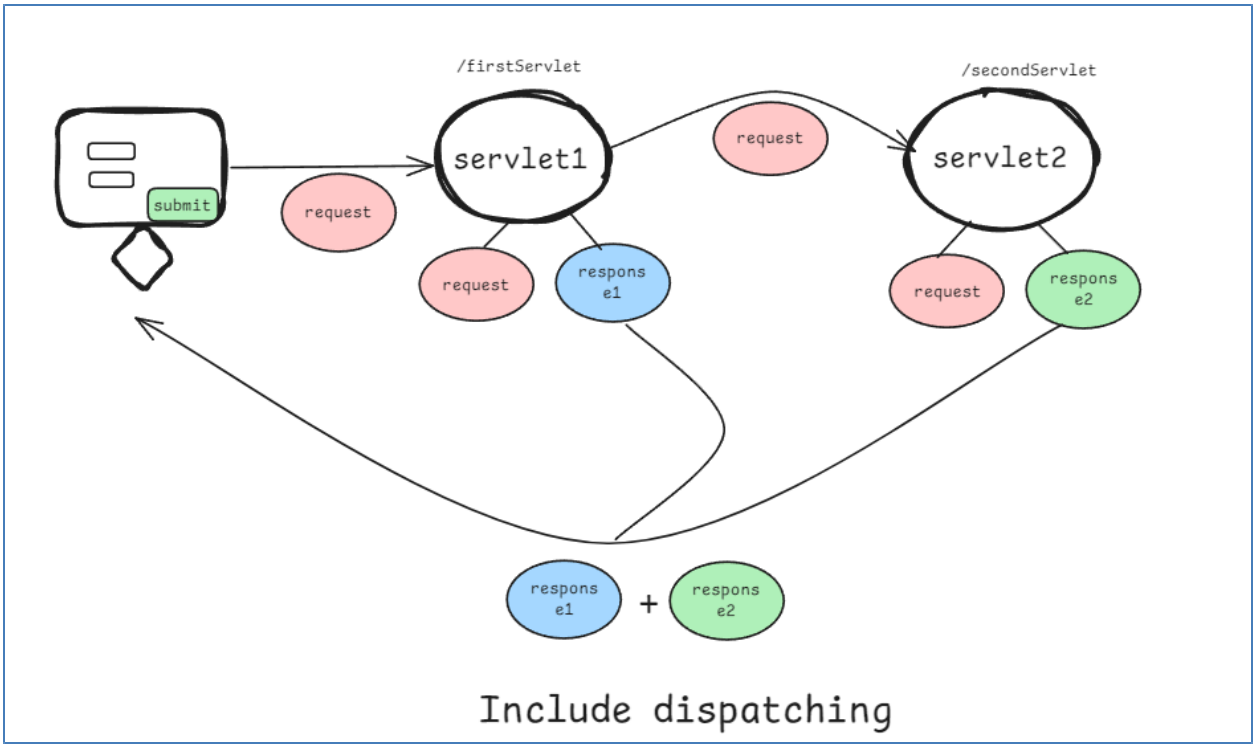
And SecondServletApp remains the same as before.

Output:


localhost:8070/FirstDynamicWebProject/
User Information
Name:
Shiva Srivastava
City:
Bangalore
Submit


localhost:8070/FirstDynamicWebProject/firstservlet
Before dispatching
Name from 2nd : Shiva Srivastava
City from 2nd: Bangalore
After response come, Name from 1st : Shiva Srivastava
After response come, dispatching City from 1st: Bangalore


Problems
Servers
Terminal
Data Source Explorer
Properties
Console
Tomcat v10.1 Server at localhost [Apache Tomcat] C:\Users\ASUS1\Downloads\eclipse-jee-2024-06-R-win32-x86_64\
INFO: Server startup in [1568] milliseconds
Internally constructor called
SecondServletApp created
execution come here



- The response will include content from both FirstServletApp and SecondServletApp, as the control returns to FirstServletApp after include().
- **Before include()**: Any content written to the response prior to calling include() will be retained and included in the final output along with the included servlet's content.
- **After include()**: Code after the include() method will continue to execute, and its output will also be added to the response, resulting in a combined response from both the original servlet and the included servlet.

👉 Summary:

- **forward():**
  - Transfers control to another servlet.
  - Response is generated by the forwarded servlet only.

    - No further response content is added after forward().
- **include():**
    - Temporarily transfers control to another servlet.
    - Combined response from both servlets is returned to the client.
    - The control returns to the original servlet after the included servlet completes its task.

This way, forward() is used when you want another resource to completely handle the request, and include() is used when you want to include additional content from another resource in your response.